

Uart_Esp32 - Projektdokumentation - Firmware

Image-Transfer-Durchsatzoptimierung | Version v0.19

Projektname	Uart_Esp32
Dokumentenart	Projektdokumentation
Thema	UART Image-Transfer: CRC32, RX-Puffer, RX-Pfad, BX3-Blockhelfer und HTTP-Festlaengenstream
Erstellungsdatum	2026-09-21
Autor / verantwortliche Person	OpenAI ChatGPT (technische Bearbeitung)
Version	v0.19
Status	Entwurf - Quellcode-/JavaScript-/CRC32-/Shell-Helper-Pruefungen erfolgreich; PlatformIO-Build und Hardwaretest ausstehend
Aenderungsdatum	2026-09-21
Ablageort	Projektordner Uart_ESP32/docs
Referenzen	BX3_ESP32_Login_Leitfaden.pdf; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf; Projektstand v0.18

Aenderungshistorie

Version	Datum	Bearbeiter	Status	Aenderung
v0.17	2026-09-21	OpenAI ChatGPT	Entwurf	Kopierfreundliche Konsole und blockweiser BX3 UART Image-Transfer.
v0.18	2026-09-21	OpenAI ChatGPT	Entwurf	Browser-/ESP32-Synchronisierung, idempotente ACKs, Fehleraufschliessung und konsolengesteuerter Neustart.
v0.19	2026-09-21	OpenAI ChatGPT	Entwurf	Ausgewaehlte Durchsatzmassnahmen 4, 6, 7, 9 und 11: CRC32 pro Block, 16-KiB-RX-Puffer, schneller RX-Pfad, persistenter BX3-Blockhelfer und HTTP-Festlaengenstream.

1. Ziel der Version v0.19

Version v0.19 setzt die vom Projekt festgelegten Beschleunigungsmassnahmen **4, 6, 7, 9 und 11** um, ohne die grundlegende Transferarchitektur aus v0.18 zu ersetzen.

Die ausgewaehlten Massnahmen sind:

1. **Massnahme 4:** CRC32 pro Block statt SHA-256 pro Block.
2. **Massnahme 6:** groesserer Hardware-UART-RX-Puffer.
3. **Massnahme 7:** effizienteres Leeren des UART-Empfangspfads waehrend des Image-Transfers.
4. **Massnahme 9:** Shell-Kommandos pro Block reduzieren.
5. **Massnahme 11:** bereits validierte Bloেকে mit fester HTTP-Laenge direkt zum Browser streamen.

Nicht Bestandteil dieser Version sind eine hoehere Baudrate, ein binaeres UART-Protokoll ohne Base64, Pipelining mehrerer Bloেকে oder eine SD-Karte. Die 32-KiB-Blockgrosse und Base64-Uebertragung bleiben daher erhalten.

2. Massnahme 4 - CRC32 pro Block

Bis v0.18 wurde fuer jeden 32-KiB-Block auf dem BX3 sha256sum ausgefuehrt und der ESP32 berechnete ebenfalls einen SHA-256 ueber den Block.

v0.19 verwendet fuer die Blockpruefung stattdessen POSIX cksum:

```
cksum /tmp/bx3blk
-> <CRC32 dezimal> <Byteanzahl> /tmp/bx3blk
```

Der ESP32 implementiert denselben POSIX-CRC32-Algorithmus mit dem Polynom 0x04C11DB7, der Einbeziehung der Blocklaenge und dem abschliessenden Einerkomplement.

Der Blockablauf lautet damit:

```
BX3: dd -> cksum -> base64
      |      |
      |      +--> Nutzdaten
      +-----> CRC32 + Byteanzahl
```

```
ESP32: Base64 dekodieren -> Laenge pruefen -> POSIX CRC32 berechnen -> vergleichen
```

Wichtig: Die Gesamtpruefung des vollstaendigen Images bleibt SHA-256. Nur die per-Block-Pruefung wurde auf CRC32 umgestellt.

Neue Marker:

```
<<<BX3IMG:BEGIN:000017>>>
<<<BX3IMG:SIZE:32768>>>
<<<BX3IMG:CRC32:1234567890>>>
...
<<<BX3IMG:END:000017>>>
```

Die Status-API liefert zusaetzlich `error_crc`. Die Weboberflaeche zeigt CRC32-Fehler getrennt von Fehlern der abschliessenden Gesamt-SHA256-Pruefung.

3. Voraussetzung `cksum` auf dem BX3

Der optimierte Transfer setzt das Kommando `cksum` in der BX3-Shell voraus. Beim Transferstart prueft die Firmware:

```
command -v cksum
```

Ist das Werkzeug vorhanden, wird der Blockhelfer installiert und der Transfer gestartet. Fehlt `cksum`, erscheint der Marker:

```
<<<BX3IMG:SETUP:CKSUM_MISSING>>>
```

Der Image-Transfer wird dann kontrolliert beendet. Wie in v0.18 wird die UART-Konsole wieder freigegeben und ein neuer Transfer bleibt bis zur erneuten Konsolenbestaetigung gesperrt.

4. Massnahme 6 - UART-RX-Puffer auf 16 KiB

Vor `HardwareSerial.begin()` wird jetzt auf dem verwendeten Hardware-UART aufgerufen:

```
_serial->setRxBufferSize(16384);
```

Die Puffervergroesserung erfolgt vor der UART-Initialisierung. Gegenueber einem kleinen Standardpuffer entsteht damit deutlich mehr Reserve, wenn Webserver, Browserstatus oder andere ESP32-Aufgaben kurzfristig CPU-Zeit benoetigen.

Die Aenderung betrifft den Hardware-RX-Ringpuffer. Der 32-KiB-Imageblockpuffer und der 8192-Byte-Konsolenpuffer bleiben davon getrennt.

5. Massnahme 7 - effizienterer UART-RX-Pfad

Im normalen UART-Basisbetrieb bleibt das bisherige Verarbeitungslimit von 512 Bytes pro `loop()`-Durchlauf erhalten.

Waehren eines aktiven Image-Transfers steigt das RX-Drain-Budget auf:

```
8192 Bytes pro loop()-Durchlauf
```

Damit werden Base64-Bursts schneller aus dem Hardware-Ringpuffer entnommen. Da waehrend des Image-Transfers RX exklusiv dem Image-Parser gehoert, entstehen in diesem Pfad keine Kopien in Rohdaten-, Konsolen- oder Decoderpuffer.

Zusaetzlich reserviert die Zeilenvariable des Image-Parsers ihre typische Kapazitaet vorab, um wiederholte dynamische Speicherallokationen bei den kurzen Base64-Zeilen zu reduzieren.

6. Massnahme 9 - permanenter BX3-Blockhelfer

Bis v0.18 uebertrug der ESP32 fuer jeden Block eine lange Shell-Befehlsfolge mit `rm`, `dd`, `wc`, `sha256sum`, `base64` und Markerausgaben.

v0.19 installiert nach der MTD-Ermittlung einmalig die Shell-Funktion:

```
bx3img_block N
```

Die Funktion ist fuer die gewaehlte Imagequelle und 32-KiB-Blockgroesse vorbereitet. Fuer jeden weiteren Block sendet der ESP32 nur noch beispielsweise:

```
bx3img_block 117
```

Innerhalb der Funktion werden ausgefuehrt:

```
dd
cksum
base64
rm
```

cksum liefert sowohl CRC32 als auch Byteanzahl. Dadurch entfallen wc und der per-Block-Aufruf von sha256sum.

Das reduziert die ueber UART zu sendende Shell-Befehlsmenge sowie den Prozess-/Parsing-Aufwand pro Block. Der abschliessende sha256sum <MTD-Device> fuer das Vollimage bleibt erhalten.

7. Massnahme 11 - HTTP-Festlaengenstream

Bis v0.18 lieferte /uart/image/block den validierten ESP32-Block als HTTP-Chunked-Response.

v0.19 kennt die bereits validierte Blocklaenge und erstellt deshalb eine Response mit fester Content-Length:

```
Content-Type: application/octet-stream
Content-Length: <validierte Blocklaenge>
X-UART-Image-Block: <N>
X-UART-Image-Length: <validierte Blocklaenge>
```

Die Response liest direkt aus dem bereits vorhandenen Imageblockpuffer des ESP32. Es wird keine Base64- oder Textkonvertierung mehr auf der HTTP-Seite vorgenommen.

Der Browser uebernimmt weiterhin genau einen validierten Block als Uint8Array und bestaetigt ihn danach ueber das blocknummerngebundene, idempotente ACK aus v0.18.

8. Unveraenderte v0.18-Synchronisierung

Die folgenden Sicherheits- und Synchronisationsfunktionen bleiben erhalten:

- getrennte Werte fuer Empfangsblock, browserbereiten Block und letztes ACK,
- HTTP 202 bei einem noch nicht browserbereiten Block,
- blocknummerngebundene idempotente ACKs,
- maximal 5 Wiederholungen pro Block,
- exklusiver UART-Besitz waehrend des Image-Transfers,
- Konsolenfreigabe nach Abbruch oder nicht wiederherstellbarem Fehler,
- erneuter Image-Start erst nach manuell bestaetigtem BX3-Shell-Zugriff in der UART-Konsole,
- kompletter Neustart ab Block 0 nach einer Unterbrechung.

9. Integritaetskonzept v0.19

Die Integritaetspruefung ist jetzt zweistufig:

Ebene	Verfahren	Zweck
Jeder 32-KiB-Block	POSIX CRC32 via cksum	Schnelle Erkennung von Uebertragungs-/Dekodierfehlern.
Gesamtes Vollimage	SHA-256 auf BX3 und ESP32	Starke End-to-End-Pruefung des vollstaendigen Images.

Bei einem Teiltransfer ab einem Startblock groesser 0 bleibt wie bisher keine Gesamt-SHA256-Pruefung verfuegbar, weil der ESP32 nicht das komplette Image ab Byte 0 empfaengt.

10. Erwartete Wirkung und Grenzen

Die Aenderungen verringern Rechen- und Shell-Overhead und vergroessern die RX-Reserve. Die absolute UART-Leitungsrate bleibt jedoch unveraendert, solange die Baudrate nicht erhoehrt wird.

Base64 bleibt ebenfalls Bestandteil des Protokolls und vergrössert die uebertragene Datenmenge weiterhin. Daher ist v0.19 eine Optimierung des bestehenden sicheren Version-1-Verfahrens und noch kein Hochgeschwindigkeits-Binaertransfer.

Die groesste praktische Verbesserung wird erwartet durch:

- weniger aufwendige Integritaetsberechnung pro Block,
- weniger lange Shell-Kommandos zwischen den Bloecken,
- weniger Risiko, dass Base64-Bursts den UART-RX-Puffer ueberlaufen,
- weniger HTTP-Framing pro Browserblock.

11. Geaenderte Dateien

Datei	Aenderung
src/UartManager.h	CRC32-Zustaende, 16-KiB-RX-Puffer, 8192-Byte-Drain-Budget, Blocklaengen-Accessor und Shell-Setup-Zustand.
src/UartManager.cpp	POSIX-CRC32, cksum-Blockhelfer, schneller RX-Pfad, grosserer UART-RX-Puffer und CRC32-Fehlerzaehler.
src/main.cpp	Firmwareanzeige v0.19, CRC32-Statusanzeige und HTTP-Response mit fester Blocklaenge.
src/ConfigDefaults.h	Firmwareversion auf 0.19.0 angehoben.
README.md	v0.19-Aenderungen und Voraussetzungen dokumentiert.
docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.19.md	Projektdokumentation.
docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.19.docx	Bearbeitbare Dokumentation.
docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.19.pdf	Vorrangige PDF-Dokumentation.
docs/2026-09-21 - Uart_Esp32 - Commit-Nachricht - Image-Transfer-Durchsatz - v0.19.txt	Verpflichtende englische Commit-Nachricht.
VALIDATION_v0.19.txt	Validierungsstand.

12. Validierung

Durchgefuehrt wurden:

- Quellcode-Konsistenzpruefung auf entfernte per-Block-SHA-Zustaende und neue CRC32-Zustaende,
- JavaScript-Syntaxpruefung aller vier eingebetteten Webskripte mit `node --check`,
- Abgleich der verwendeten ESPAsyncWebServer-API fuer Callback-Responses mit fester Content-Length gegen die im Projekt enthaltene Bibliotheksversion,
- eigenstaendiger C++17-Test der implementierten POSIX-CRC32-Funktion gegen das Systemkommando `cksum` mit einem 32-KiB-Zufallsblock,
- Shell-Syntaxtest des neuen `bx3img_block`-Helpers mit einem 32-KiB-Testblock,
- ZIP-Struktur- und Patch-Pruefung der Auslieferung,
- visuelle Renderpruefung von DOCX und PDF.

Noch ausstehend:

- echter PlatformIO-Build,
- Flash-Test auf dem Ziel-ESP32,
- Pruefung, dass `cksum` auf dem konkreten BX3-System vorhanden ist,
- realer 16-MiB-End-to-End-Transfer,
- Vergleich der realen Transferrate v0.18 zu v0.19,

- unabhaengiger SHA-256-Vergleich der fertigen Image-Datei auf dem PC.

13. Testcheckliste am BX3

- ☐ Firmware lokal mit `pio run -e uart_esp32_usb` bauen.
- ☐ UART-Konfiguration und TX-Freigabe pruefen.
- ☐ In der BX3-Shell `command -v cksum` testen.
- ☐ Image-Transfer starten und auf BX3-Blockhelfer bereit achten.
- ☐ CRC32-Fehlerzaehler beobachten.
- ☐ Transferrate ueber mindestens 50 bis 100 Bloecke notieren.
- ☐ Einen Transfer absichtlich unterbrechen und die v0.18-Konsolenfreigabe/Neustartsperrung pruefen.
- ☐ Volltransfer ab Block 0 durchlaufen lassen.
- ☐ BX3- und ESP32-Gesamt-SHA256 in der Weboberflaeche vergleichen.
- ☐ Heruntergeladenes Image auf dem PC erneut mit SHA-256 pruefen.

14. Hinweise zur Hardware

Die bestehenden Hinweise des BX3-Konsolenleitfadens bleiben unveraendert: gemeinsame Masse, Logikpegel vor aktivem TX pruefen, BX3-VCC nicht mit der ESP32-Versorgung verbinden und waehrend der Erprobung geeignete Schutzmassnahmen in der TX-Leitung verwenden.

Die v0.19-Aenderungen betreffen ausschliesslich Firmware, UART-Pufferung, Blockintegritaet, Shell-Steuerung und HTTP-Ausgabe.